

**UNIVERSIDAD MAYOR REAL Y PONTIFICIA
DE SAN FRANCISCO XAVIER DE CHUQUISACA**

*Facultad de Ciencia y Tecnología
Ingeniería en Ciencias de la Computación*

PRÁCTICA Nº 7

**Elaboración de un Plan de
Calidad de Software (SQAP)**

conforme al estándar IEEE Std 730

Sucre — Bolivia

1. Información general de la práctica

Campo	Detalle
Práctica Nº	7
Título	Elaboración de un Plan de Calidad de Software (SQAP) conforme a IEEE Std 730
Asignatura	COM450 — Calidad de Software
Carrera	Ingeniería en Ciencias de la Computación
Gestión	I / 2026
Modalidad	Grupal — 3 a 4 estudiantes por equipo
Prerrequisito aplicado	Arquitectura de Software (atributos de calidad, tácticas, escenarios)
Ubicación en el cronograma	Tras el Tema 3 (Procesos de Gestión de Calidad), antes de las prácticas de JUnit, SonarQube, Cypress y Jenkins
Entregable final	Documento SQAP, conforme a las 16 secciones del IEEE Std 730-2002, aplicado a un proyecto piloto

2. Objetivos de la práctica

2.1 Objetivo general

Elaborar un Plan de Aseguramiento de Calidad de Software (SQAP) conforme al estándar IEEE Std 730, aplicado a un proyecto de desarrollo pequeño, integrando los conceptos de modelos de calidad, de modo que el documento sirva como contrato interno de calidad y como insumo para las prácticas posteriores de verificación y validación.

2.2 Objetivos específicos

- Identificar los atributos de calidad del producto software aplicables según ISO/IEC 9126 (funcionalidad, eficiencia, fiabilidad, usabilidad, seguridad, mantenibilidad, portabilidad, compatibilidad).
- Definir métricas cuantitativas y cualitativas, con valores objetivo realistas y trazables a requerimientos no funcionales.
- Seleccionar y justificar estándares de codificación, convenciones de nomenclatura y buenas prácticas aplicables al stack tecnológico del proyecto piloto.
- Planificar revisiones técnicas, inspecciones de código y auditorías en los puntos de control del ciclo de vida.
- Diseñar el flujo de gestión de defectos con clasificación por severidad, prioridad y criterios SLA de corrección.
- Construir una matriz de riesgos de calidad con probabilidad, impacto y plan de mitigación.
- Estructurar un documento profesional conforme a las 16 secciones normativas del IEEE 730-2002, adoptando las tres áreas del IEEE 730-2014 (implementación del proceso SQA, aseguramiento del producto y aseguramiento del proceso).

2.3 Resultados de aprendizaje

Al concluir la práctica, el estudiante será capaz de:

1. Redactar un SQAP conforme a IEEE 730 para un proyecto de software real para el software asignado.
2. Justificar decisiones de calidad anclándolas en modelos normativos (ISO 9126, CMMI, TMMi, SWEBOOK V4).
3. Articular el plan de calidad con la arquitectura (prerrequisito) mostrando cómo las tácticas arquitectónicas se verifican mediante métricas y pruebas.
4. Usar el SQAP como instrumento rector durante el desarrollo, no como documento ceremonial de cierre.

3. Fundamento teórico

3.1 ¿Qué es un Plan de Calidad de Software?

Un Plan de Aseguramiento de Calidad de Software, conocido por su sigla inglesa SQAP (Software Quality Assurance Plan), es el documento maestro que establece los compromisos, actividades, responsables, estándares y métricas con los que un proyecto de desarrollo garantizará que el producto resultante cumpla los requerimientos de calidad acordados. A diferencia de un plan de pruebas (que se limita a la verificación ejecutable) o un plan de proyecto (que se centra en alcance, tiempo y costo), el SQAP orquesta todas las actividades de aseguramiento de calidad a lo largo del ciclo de vida completo.

Principio rector — IEEE Std 730-2014:

El aseguramiento de calidad debe implementarse bajo el paradigma de quality by design y left-shift for quality engineering, desplazando la detección y prevención de defectos hacia las fases tempranas del ciclo de vida, donde el costo de corrección es hasta 100 veces menor que en producción (SWEBOK V4, capítulo 10).

3.2 Estándar normativo de referencia

El estándar internacional que rige la estructura y contenido del SQAP es el IEEE Std 730, emitido por la IEEE Computer Society. Existen dos versiones vigentes y complementarias:

Versión	Enfoque	Uso en la práctica
IEEE 730-2002	Plantilla prescriptiva de 16 secciones. Didáctica y autoexplicativa.	Esqueleto del documento a entregar.
IEEE 730-2014	Marco de procesos armonizado con ISO/IEC/IEEE 12207 e ISO/IEC/IEEE 15289. Tres áreas: implementación del proceso SQA, aseguramiento del producto, aseguramiento del proceso.	Marco conceptual de agrupación y justificación de decisiones.

3.3 Normas complementarias

El SQAP se conecta con otras normas que el estudiante ya conoce de los temas previos:

- ISO/IEC 9126 — fuente de los atributos de calidad del producto y calidad en uso.

- ISO/IEC/IEEE 12207 — procesos del ciclo de vida del software en los que se insertan las actividades SQA.
- ISO/IEC 29110 — perfil para entidades muy pequeñas (VSE), aplicable directamente a proyectos estudiantiles y startups.
- CMMI-DEV — áreas de proceso PPQA (Product and Process Quality Assurance), VAL (Validation) y VER (Verification).
- TMMi — modelo de madurez de pruebas, complementario a CMMI.
- SWEBOK V4, capítulo 10 — cuerpo de conocimiento sobre calidad de software.
- IEEE 1028 — revisiones e inspecciones de software.
- IEEE 829 / ISO 29119 — documentación de pruebas.

3.4 Las 16 secciones del SQAP según IEEE 730-2002

El estándar define una tabla de contenido canónica que todo SQAP conforme debe cubrir. Si alguna sección no aplica al proyecto, debe dejarse explícita la frase "This section is not applicable to this plan" junto con la justificación, pero no puede omitirse.

Nº	Sección	Qué debe responder
1	Propósito	Qué software cubre el plan, alcance, porción del ciclo de vida abarcada, nivel de criticidad
2	Documentos de referencia	Normas aplicables, SRS, documento de arquitectura, contrato, regulaciones
3	Gestión (Management)	Estructura organizacional, roles SQA, responsabilidades, independencia del equipo de calidad
4	Documentación	Artefactos documentales a producir, calendario de revisión, criterios de adecuación
5	Estándares, prácticas, convenciones y métricas	Estándares de código, convenciones, métricas con valores objetivo, forma de monitoreo
6	Revisiones de software	Revisiones técnicas, inspecciones Fagan, walkthroughs, auditorías
7	Pruebas (Test)	Estrategia y niveles de prueba, criterios de entrada y salida, cobertura
8	Reporte de problemas y acción correctiva	Flujo de defectos, clasificación, SLA, herramienta de gestión
9	Herramientas, técnicas y metodologías	Herramientas de desarrollo, QA, CI/CD, técnicas específicas (TDD, BDD, pair programming)

Nº	Sección	Qué debe responder
10	Control de medios (Media control)	Control de repositorios, backups, acceso físico y lógico
11	Control de proveedores	Calidad de librerías y componentes de terceros, supply chain security
12	Registros: recolección, mantenimiento y retención	Evidencias a conservar, período de retención, ubicación
13	Capacitación (Training)	Formación necesaria del equipo en herramientas, estándares y dominio
14	Gestión de riesgos	Identificación, análisis, mitigación y monitoreo de riesgos de calidad
15	Glosario	Términos específicos del proyecto y del dominio
16	Procedimiento de cambio e historia del SQAP	Control de versiones del propio plan

4. Proyecto piloto

El SQAP debe aplicarse a un proyecto piloto concreto asignado a cada grupo,. Cada equipo elegirá uno de los siguientes sistemas como objeto de su plan.

4.1 Opciones de proyecto

Opción	Descripción	Stack sugerido
A. Gestión de biblioteca universitaria	CRUD web con autenticación, préstamos, multas y reportes. Roles: estudiante, docente, bibliotecario, administrador.	Java + Spring Boot + PostgreSQL + React, o PHP + Laravel + MySQL + Blade
B. API REST de inventario de laboratorio	Catálogo de equipos, reservas, reporte de daños, trazabilidad. Documentación OpenAPI.	Node.js + Express + MongoDB, o Spring Boot + PostgreSQL
C. Control de asistencia móvil	App cliente con escaneo QR o geolocalización, backend para generación de reportes y dashboard administrativo.	Flutter / Android nativo + Firebase o backend propio en Node/Spring
D. Registro de calificaciones	Web para docentes, con cálculo automático de promedios, exportación a PDF, historial y auditoría.	Laravel + MySQL + Bootstrap, o Django + PostgreSQL
E. Punto de venta pequeño (POS)	Venta, inventario, cierre de caja, reportes diarios. Multiusuario con control de permisos.	Laravel + MySQL, o Django + PostgreSQL

4.2 Proyecto a realizar

- Tamaño: entre 2 y 4 historias de usuario o casos de uso principales del proyecto asignado.
- Complejidad: al menos 1 roles de usuario
- Verificabilidad: los atributos de calidad propuestos deben poder medirse con las herramientas disponibles (JUnit, Cypress, JMeter, SonarQube).

5. Desarrollo de la práctica

La práctica se estructura en cuatro entregables parciales y un entregable final consolidado. Esta división permite retroalimentación temprana y evita el patrón antipedagógico de "todo al final". Cada entregable agrupa varias de las 16 secciones del IEEE 730 según afinidad temática.

5.1 Entregable 1 — Fundamentos del plan

Secciones IEEE 730 cubiertas

Sección 1 — Propósito

Sección 2 — Documentos de referencia

Sección 3 — Gestión (roles y organización)

Sección 15 — Glosario

Sección 16 — Procedimiento de cambio e historia

Contenido mínimo exigido:

- Identificación del proyecto piloto, alcance claramente delimitado, porción del ciclo de vida que cubre el SQAP (ej. requerimientos, diseño, construcción, pruebas).
- Nivel de criticidad del software según integridad (no crítico, de apoyo, crítico para el negocio o crítico para seguridad).
- Lista de documentos de referencia: SRS del proyecto, documento de arquitectura (SAD), estándares adoptados, contratos internos del equipo.
- Organigrama del equipo con roles explícitos: Líder de Proyecto, Arquitecto, Desarrolladores, Responsable SQA, Tester. Se valora que el rol SQA tenga cierta independencia del rol Desarrollador para evitar conflicto de intereses.
- Matriz RACI (Responsible, Accountable, Consulted, Informed) para las actividades de calidad.
- Glosario del proyecto con al menos 20 términos técnicos y del dominio.
- Procedimiento de cambio del propio SQAP: quién puede proponer cambios, cómo se aprueban, tabla de historial de versiones.

5.2 Entregable 2 — Aseguramiento del producto

Secciones IEEE 730 cubiertas

Sección 4 — Documentación

Sección 5 — Estándares, prácticas, convenciones y métricas

Sección 7 — Pruebas

Contenido mínimo exigido:

- Lista de documentos que el proyecto producirá: SRS, SAD, manual de usuario, manual técnico, reporte de pruebas, actas de revisión. Para cada documento: responsable, fecha prevista, criterios de adecuación, tipo de revisión.
- Estándares de codificación adoptados con justificación (ej. Google Java Style Guide, PSR-12 para PHP, PEP 8 para Python, Airbnb JavaScript Style Guide).
- Convenciones de nomenclatura para variables, clases, tablas, endpoints.
- Tabla de atributos de calidad ISO/IEC 25010 relevantes para el proyecto, con al menos dos métricas por cada atributo seleccionado y valores objetivo numéricos.
- Estrategia de pruebas por nivel: unitarias, integración, sistema, aceptación. Para cada nivel: técnica, herramienta, responsable, criterio de entrada y criterio de salida.
- Cobertura mínima exigida (ej. 80% de líneas en módulos críticos, 60% global).

Ejemplo de tabla de métricas — atributo Fiabilidad (ISO 9126)

Subcaracterística	Métrica	Fórmula / Instrumento	Valor objetivo
Madurez	Densidad de defectos en producción	Defectos / KLOC durante los primeros 30 días	≤ 1.0 def/KLOC
Disponibilidad	Uptime mensual	$(\text{tiempo total} - \text{tiempo caído}) / \text{tiempo total}$	$\geq 99.0\%$
Tolerancia a fallos	Tasa de recuperación	% de operaciones que reintentan con éxito	$\geq 95\%$
Capacidad de recuperación	RTO (Recovery Time Objective)	Tiempo máximo de restauración tras fallo	≤ 30 minutos

Plazo sugerido: semana 2 tras el inicio de la práctica.

5.3 Entregable 3 — Aseguramiento del proceso

Secciones IEEE 730 cubiertas

Sección 6 — Revisiones de software

Sección 8 — Reporte de problemas y acción correctiva

Sección 9 — Herramientas, técnicas y metodologías

Sección 13 — Capacitación

Contenido mínimo exigido:

- Calendario de revisiones a lo largo del ciclo de vida: revisión de requerimientos (SRR), revisión de arquitectura (ADR), revisión de diseño detallado (DDR), revisiones de código continuas (peer review), auditoría de cierre.
- Para cada tipo de revisión: quién la ejecuta, cuándo, qué artefacto revisa, lista de verificación (checklist) y criterios de aceptación.
- Adopción explícita de inspecciones formales tipo Fagan para artefactos críticos (al menos SRS y diseño arquitectónico).
- Flujo completo de gestión de defectos: estados (nuevo, asignado, en progreso, resuelto, verificado, cerrado, reabierto), transiciones permitidas, responsables por estado.
- Clasificación de severidad (crítica, alta, media, baja) con definición precisa de cada nivel, y tiempos SLA de respuesta y resolución para cada combinación severidad × prioridad.
- Plantilla de reporte de defecto: título, pasos para reproducir, comportamiento esperado, comportamiento observado, entorno, capturas, severidad, prioridad.
- Stack completo de herramientas: gestión de requerimientos, control de versiones, CI/CD, análisis estático, pruebas unitarias, pruebas E2E, pruebas de carga, gestión de defectos, comunicación.
- Plan de capacitación: competencias requeridas por rol, brechas detectadas en el equipo, sesiones de capacitación planificadas.

Ejemplo de SLA de gestión de defectos

Severidad	Descripción	Tiempo de respuesta	Tiempo de resolución
Crítica (S1)	El sistema es inusable o hay pérdida de datos	1 hora	8 horas

Severidad	Descripción	Tiempo de respuesta	Tiempo de resolución
Alta (S2)	Funcionalidad principal afectada sin workaround	4 horas	24 horas
Media (S3)	Funcionalidad secundaria afectada o existe workaround	1 día hábil	1 semana
Baja (S4)	Defecto cosmético o mejora menor	3 días hábiles	Próximo sprint

5.4 Entregable 4 — Gobernanza y riesgos

Secciones IEEE 730 cubiertas

Sección 10 — Control de medios

Sección 11 — Control de proveedores

Sección 12 — Registros: recolección, mantenimiento y retención

Sección 14 — Gestión de riesgos

Contenido mínimo exigido:

- Política de control del repositorio: estrategia de branching (Git Flow, Trunk-Based, GitHub Flow), reglas de protección de rama principal, política de pull requests, política de merge.
- Política de backups: qué se respalda, frecuencia, retención, responsable, prueba periódica de restauración.
- Control de acceso: quién tiene permisos de lectura, escritura, administración sobre repositorio, ambientes y herramientas.
- Gestión de dependencias de terceros: criterios de selección (licencia, mantenimiento activo, comunidad), análisis de vulnerabilidades (OWASP Dependency-Check, Snyk, GitHub Dependabot), revisión de licencias (MIT, Apache 2.0, GPL, propietarias).
- Matriz de componentes de terceros con columnas: nombre, versión, licencia, justificación de uso, responsable de actualización, última revisión de seguridad.
- Catálogo de evidencias de calidad a preservar: actas de revisión, reportes de pruebas, logs de pipeline CI, métricas de Sonar, reportes de JMeter. Para cada uno: ubicación, formato, período de retención.
- Matriz de riesgos de calidad con al menos 10 riesgos identificados, cada uno con probabilidad, impacto, exposición (prob × impacto), plan de mitigación, responsable y plan de contingencia.

Ejemplo de matriz de riesgos

Riesgo	Prob.	Impacto	Exposición	Mitigación
Rotación de miembros del equipo durante el semestre	Alta	Alto	Alta	Documentación actualizada, pair programming, backup de conocimiento
Requerimientos ambiguos del cliente académico	Alta	Medio	Alta	Revisiones tempranas de SRS, prototipos validados
Deuda técnica acumulada por presión de cronograma	Media	Alto	Alta	Quality gate de SonarQube bloqueante, refactoring continuo
Falla del servidor de integración continua	Baja	Medio	Media	Respaldo de configuración, runner alternativo, documentación de setup
Librerías de terceros con vulnerabilidades críticas	Media	Alto	Alta	Análisis Dependabot en cada PR, actualización mensual

5.5 Entregable final consolidado

Documento único PDF y DOCX que integre los cuatro entregables parciales, ya corregidos según la retroalimentación, con portada institucional, tabla de contenidos, numeración consistente, lista de tablas y figuras, y anexos.

El documento debe presentarse acompañado de:

- Presentación oral grupal de 20 minutos que defienda las decisiones clave del plan.
- Repositorio Git del proyecto piloto con el SQAP versionado en la carpeta docs/.
- Plantilla de reporte de defectos como archivo independiente (Markdown o PDF).
- Hoja de cálculo con la matriz de riesgos actualizable.

6. Ejemplo conductor

Para orientar a los equipos, a continuación se presenta el contenido esquemático de las tres primeras secciones del SQAP aplicado al Proyecto A (Gestión de biblioteca universitaria). El propósito es ilustrar el nivel de detalle esperado, no servir como respuesta completa.

Sección 1 — Propósito (ejemplo esquemático)

Proyecto BIBLIOUSFX v1.0

Este plan de aseguramiento de calidad cubre el desarrollo del sistema BIBLIOUSFX, una aplicación web de gestión de biblioteca para la Facultad de Ciencia y Tecnología. El SQAP abarca las fases de requerimientos, diseño, construcción y pruebas, y se aplica durante la Gestión I/2026 por el equipo M-07. El sistema se clasifica como software de apoyo (nivel de integridad 2 según IEEE 730-2014 Anexo I), dado que su falla causaría inconveniencia operativa pero no pérdida de vidas ni daño económico mayor.

Sección 3 — Gestión (ejemplo esquemático)

El equipo M-07 adopta una estructura matricial con roles explícitos y matriz RACI. El rol de Responsable SQA se asigna al miembro que no desarrolla los módulos críticos de préstamos y multas, asegurando independencia mínima de revisión.

Rol	Responsabilidad principal	Miembro asignado
Líder de Proyecto	Coordinación, comunicación con docente, cronograma	[Nombre 1]
Arquitecto de Software	Diseño arquitectónico, decisiones técnicas (ADR)	[Nombre 2]
Responsable SQA	Plan y ejecución de SQA, métricas, auditoría	[Nombre 3]
Desarrollador Frontend	Construcción de UI React, pruebas Cypress	[Nombre 4]
Desarrollador Backend	API Spring Boot, pruebas JUnit	[Nombre 1 y 2]

Sección 5 — Atributos de calidad seleccionados (ejemplo esquemático)

Del modelo ISO/IEC 25010, el equipo prioriza cinco de los ocho atributos del producto, según relevancia para el dominio de biblioteca universitaria:

Atributo (25010)	Justificación	Métrica principal	Objetivo
Adecuación funcional	Las operaciones de préstamo y devolución deben ejecutarse correctamente	Tasa de defectos en pruebas de aceptación	0 defectos críticos
Eficiencia de desempeño	Uso concurrente por estudiantes en horario pico	Tiempo de respuesta p95 para búsqueda de libro	≤ 800 ms
Fiabilidad	Indisponibilidad afecta a toda la facultad	Uptime mensual	$\geq 99.0\%$
Seguridad	Datos personales y académicos protegidos por ley boliviana	Vulnerabilidades OWASP Top 10 detectadas	0 de severidad alta o crítica
Mantenibilidad	Proyecto continuará en gestiones posteriores	Deuda técnica en SonarQube	< 5 días-persona

7. Criterios de evaluación

La práctica se evalúa sobre 100 puntos, distribuidos en siete criterios que reflejan tanto la completitud formal como la calidad técnica del plan. Cada entregable parcial recibe retroalimentación con indicación de puntos a mejorar antes de la entrega final.

Criterio	Descripción	Puntos
Completitud estructural	Las 16 secciones del IEEE 730-2002 están presentes y contienen el contenido mínimo exigido	25
Alineación con ISO/IEC 9126	Atributos y métricas bien seleccionados, trazables a requerimientos del proyecto y justificados	15
Calidad de métricas	Métricas medibles con valores objetivo realistas, instrumentos de medición definidos, sin métricas genéricas copiadas	15
Matriz de riesgos	Al menos 10 riesgos pertinentes, con probabilidad e impacto justificados y planes de mitigación accionables	10
Plan de revisiones y pruebas	Coherente con el proyecto piloto, con checklists, criterios de entrada y salida por nivel	15
Integración con el ecosistema de prácticas	El SQAP anticipa y guía explícitamente las prácticas de JUnit, SonarQube, Cypress, JMeter y Jenkins	10
Presentación profesional	Redacción clara, trazabilidad, formato consistente, bibliografía, uso correcto de normas citadas	10
Total		100

7.1 Rúbrica de niveles

Cada criterio se califica según cuatro niveles de logro:

Nivel	% asignado	Descripción
Excelente	90-100%	Supera la expectativa, con rigor y originalidad
Satisfactorio	70-89%	Cumple con lo esperado, con detalles menores corregibles
En desarrollo	50-69%	Contenido presente pero incompleto o parcialmente inadecuado

Nivel	% asignado	Descripción
Insuficiente	0-49%	Contenido ausente, inadecuado o copiado sin adaptación

8. Recursos y formatos de referencia

8.1 Estándares normativos

- IEEE Std 730-2014 — Software Quality Assurance Processes. Disponible en IEEE Xplore (standards.ieee.org/ieee/730/5284/).
- IEEE Std 730-2002 — Software Quality Assurance Plans. PDF docente disponible en www.cs.utep.edu/isalamah/courses/5387/IEEE-Std-730-2002.pdf
- ISO/IEC 25010 — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
- ISO/IEC/IEEE 12207 — Systems and software engineering — Software life cycle processes.
- ISO/IEC 29110 — Systems and software engineering — Lifecycle profiles for Very Small Entities.
- IEEE Std 1028 — Software Reviews and Audits.
- ISO/IEC/IEEE 29119 — Software testing (serie completa).

8.2 Plantillas y ejemplos

- Plantilla SQAP de la Universidad de Kansas (Prof. Saiedian): people.eecs.ku.edu/~saiedian/Teaching/StdS/0730.pdf
- Texto íntegro IEEE 730-2014 (repositorio académico): www.yegor256.com/pdf/ieee-730-2014.pdf
- Plantillas universitarias en español buscables como "plantilla plan aseguramiento calidad software filetype:pdf" en Google Scholar (Universidad de Sevilla, UPM, UTN Argentina tienen versiones docentes públicas).

8.3 Bibliografía complementaria

- Pressman, R. S. & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9ª ed.). McGraw-Hill. — Capítulos 17 y 18 sobre calidad y SQA.
- Galin, D. (2018). Software Quality: Concepts and Practice. Wiley-IEEE Press.
- Sommerville, I. (2015). Software Engineering (10ª ed.). Pearson. — Capítulo 24 sobre gestión de calidad.
- SWEBOK V4 (2024). Software Engineering Body of Knowledge. IEEE Computer Society. — Capítulo 10.
- ISTQB Certified Tester Foundation Level Syllabus v4.0 (2023). International Software Testing Qualifications Board.

8.4 Herramientas sugeridas para el desarrollo del SQAP

- Editor de documento: Microsoft Word, Google Docs o LaTeX (plantilla IEEE). Recomendado: Word con plantilla Midnight Executive del curso.
- Diagramas: draw.io, Lucidchart, Mermaid, PlantUML.
- Matriz de riesgos: Excel, Google Sheets o plantilla Trello Risk Board.
- Gestión de SQAP y artefactos: repositorio Git con carpeta docs/ versionada.

10. Resultados esperados

Al finalizar esta práctica, se espera que el estudiante haya desarrollado competencias profesionales concretas y verificables.

10.1 Competencias desarrolladas

- Capacidad para redactar un SQAP conforme a IEEE 730 en un proyecto real.
- Habilidad para seleccionar, justificar e instrumentar atributos de calidad según ISO/IEC 25010.
- Dominio de la articulación entre arquitectura de software y aseguramiento de calidad, aprovechando el prerrequisito del curso.
- Capacidad para planificar actividades de calidad a lo largo del ciclo de vida y no únicamente al final.
- Competencia en la identificación y gestión de riesgos de calidad.
- Fluidez en el uso de normas internacionales como instrumento de decisión técnica, no como adorno documental.

10.2 Articulación con el resto del curso

Esta práctica es el puente conceptual entre la teoría de calidad (temas 1, 2 y 3) y las prácticas de verificación y validación (prácticas 1 a 6). El SQAP elaborado aquí se convierte en el documento rector de todas las actividades posteriores:

Práctica posterior	Cómo se alimenta del SQAP
P1 — Pruebas unitarias con JUnit	Sección 7 (Test) define cobertura mínima, estándar de nombramiento de tests y estrategia AAA
P2 — TDD en Java	Sección 9 (Herramientas y metodologías) adopta TDD como práctica obligatoria en módulos críticos
P3 — SonarQube	Sección 5 (Métricas) define los valores objetivo que el Quality Gate deberá hacer cumplir
P4 — Cypress (E2E)	Sección 7 define los escenarios de aceptación automatizables
P5 — JMeter	Sección 5 fija los objetivos de eficiencia de desempeño que se validarán con carga

Práctica posterior	Cómo se alimenta del SQAP
P6 — Jenkins (CI/CD)	Sección 9 establece el pipeline como el mecanismo que orquesta el cumplimiento del plan

10.3 Impacto formativo

Más allá de las competencias técnicas, la práctica busca transformar la mentalidad del estudiante frente a la calidad: de una actitud reactiva (testear al final lo que salga) a una actitud proactiva (planificar la calidad antes de escribir la primera línea de código). Esta mentalidad es la que distingue a un ingeniero de software de un programador, y es el objetivo último de la asignatura Calidad de Software en la carrera de Ingeniería en Ciencias de la Computación.

— Fin de la Práctica 7 —